

IoT Home Control System - Setup Plan

Project Overview

This project creates a complete IoT system to control home devices (lights and fans) remotely through the internet using a Flutter mobile application. The system includes device control, voltage monitoring, and timer scheduling features.

Hardware Components Required

1. Microcontroller & Connectivity

- **ESP8266 NodeMCU** (1x) - WiFi-enabled microcontroller
 - Alternative: ESP32 (more powerful, dual-core)
 - Alternative: Arduino Uno + ESP8266 WiFi Module

2. Relays (for device control)

- **5V Relay Module** (3x) - One for each device
 - Recommended: 2-channel or 4-channel relay module
 - Ensure relays can handle your device voltage (110V/220V AC)

3. Voltage/Current Monitoring

- **ACS712 Current Sensor** (3x) - 30A version recommended
 - OR **INA219 Current/Voltage Sensor** (3x) - More accurate
 - OR **PZEM-004T Energy Meter** (3x) - Pre-built solution

4. Power Supply

- **5V Power Supply** (1x) - For ESP8266 and relays
 - USB power bank or wall adapter (2A minimum)
 - OR **12V Power Supply** if using 12V relays

5. Additional Components

- **Breadboard** (1x) - For prototyping
- **Jumper Wires** (Male-to-Male, Male-to-Female) - Multiple
- **Resistors** (220 Ω , 1k Ω) - For LED indicators (optional)
- **LEDs** (3x) - Status indicators (optional)
- **Resistors for voltage dividers** (if needed for sensors)

6. Safety Components

- **Fuse holders and fuses** - For safety
- **Enclosure box** - To house the electronics safely

- **Terminal blocks** - For secure wire connections

7. Devices to Control

- **Light Bulbs** (2x) - Standard AC bulbs
 - **Fan** (1x) - AC fan or fan with AC adapter
-

Circuit Connections

ESP8266 Pin Configuration

ESP8266 NodeMCU Pinout:

- D1 (GPIO5) → Relay 1 (Light 1)
- D2 (GPIO4) → Relay 2 (Light 2)
- D3 (GPIO0) → Relay 3 (Fan)
- A0 (ADC) → Voltage monitoring (if using voltage divider)
- D4 (GPIO2) → SDA (for INA219 sensors)
- D5 (GPIO14) → SCL (for INA219 sensors)
- 3.3V → Power for sensors
- GND → Common ground

Relay Module Connections

For each relay module:

- VCC → 5V (or 12V depending on relay)
- GND → GND
- IN1 → ESP8266 GPIO pin (D1, D2, or D3)
- COM → AC Live wire (from wall outlet)
- NO → Device Live wire (to device)
- NC → Not used (or for normally closed operation)

ACS712 Current Sensor Connections

For each ACS712 sensor:

- VCC → 5V
- GND → GND
- OUT → ESP8266 Analog pin (A0, or use analog multiplexer)
- AC Live wire passes through the sensor hole

INA219 Sensor Connections (Recommended)

For each INA219 sensor:

- VCC → 3.3V
- GND → GND
- SDA → D4 (GPIO2) - Shared I2C bus
- SCL → D5 (GPIO14) - Shared I2C bus

- Connect in parallel with device load
-

Arduino/ESP8266 Code Setup

Step 1: Install Required Libraries

Open Arduino IDE and install these libraries: 1. **ESP8266WiFi** (built-in) 2. **ESP8266WebServer** (built-in) 3. **ArduinoJson** (by Benoit Blanchon) - Version 6.x 4. **Wire** (built-in) - For I2C communication 5. **Adafruit_INA219** (if using INA219 sensors)

Step 2: Upload Arduino Code

Create a new sketch and upload the following code (see `arduino_code.ino` file in project).

Step 3: Configure WiFi

1. Upload the code to ESP8266
2. Open Serial Monitor (115200 baud)
3. The ESP8266 will create a WiFi access point
4. Connect to it and configure your home WiFi credentials
5. Note the IP address shown in Serial Monitor

Step 4: Update Flutter App IP Address

1. Open `lib/services/iot_service.dart`
2. Change `baseUrl` to your ESP8266's IP address:

```
static const String baseUrl = 'http://YOUR_ESP8266_IP';
```

Physical Setup Instructions

Step 1: Safety First

WARNING: Working with AC power is dangerous. If you're not experienced, seek professional help.

1. **Turn off all power** before making connections
2. Use proper wire gauges for your current requirements
3. Ensure all connections are secure
4. Use proper insulation and enclosures

Step 2: Assemble the Circuit

1. **Place ESP8266 on breadboard**
2. **Connect relays** to GPIO pins
3. **Connect sensors** (ACS712 or INA219)
4. **Connect power supply** to ESP8266 (5V via USB or VIN)
5. **Connect common ground** for all components

Step 3: Connect Devices

For each device (Light/Fan):

1. **AC Power Input:**
 - Connect AC Live wire to Relay COM terminal
 - Connect AC Neutral directly to device (bypass relay)
2. **Device Connection:**
 - Connect device Live wire to Relay NO (Normally Open) terminal
 - When relay activates, circuit completes and device turns ON
3. **Current Sensor:**
 - Pass the Live wire through ACS712 sensor hole
 - OR connect INA219 in series/parallel with device

Step 4: Enclosure Setup

1. Place all electronics in a proper enclosure
2. Ensure ventilation for heat dissipation
3. Use strain relief for wires entering/exiting
4. Label all connections clearly
5. Add fuse protection on AC input

Network Configuration

Option 1: Local Network (Same WiFi)

- ESP8266 and mobile device on same WiFi network
- Use local IP address (e.g., 192.168.1.100)

Option 2: Internet Access (Remote Control)

- Configure router port forwarding
- Use dynamic DNS service (e.g., No-IP, DuckDNS)
- Update Flutter app with public IP or domain

Option 3: Cloud Service (Advanced)

- Use MQTT broker (e.g., Adafruit IO, ThingsBoard)
- Modify code to use MQTT instead of HTTP

Arduino Code Structure

The Arduino code should: 1. Connect to WiFi 2. Create HTTP server endpoints: - **GET /ping** - Connection test - **POST /control** - Control device on/off - **GET /status** - Get all devices status - **GET /status/{deviceId}** - Get specific device status 3. Read sensor values (voltage, current, power) 4. Control relays based on commands 5. Return JSON responses

Flutter App Features

1. Device Control

- Turn devices ON/OFF
- Real-time status updates
- Visual feedback with icons and colors

2. Voltage Monitoring

- Real-time voltage display
- Current consumption
- Power calculation ($V \times I$)

3. Timer Scheduling

- Set timers for device control
- One-time or recurring schedules
- Enable/disable timers

4. Settings

- Configure Arduino IP address
 - Test connection
 - View connection status
-

Testing & Troubleshooting

Testing Steps

1. **Hardware Test:**
 - Upload code to ESP8266
 - Check Serial Monitor for WiFi connection
 - Test relay activation manually
2. **Network Test:**

- Ping ESP8266 IP from computer
 - Access `http://ESP8266_IP/ping` in browser
 - Should return success message
3. **App Test:**
- Update IP in Flutter app
 - Run app and test connection
 - Try toggling devices
 - Check voltage readings

Common Issues

1. **Cannot Connect:**
 - Check WiFi credentials
 - Verify IP address
 - Ensure same network
 - Check firewall settings
2. **Relay Not Working:**
 - Verify GPIO pin connections
 - Check relay power supply
 - Test relay with multimeter
3. **Sensor Readings Wrong:**
 - Calibrate sensors
 - Check wiring
 - Verify sensor power supply
 - Check ADC reference voltage
4. **App Crashes:**
 - Check network permissions in `AndroidManifest.xml`
 - Verify JSON parsing
 - Check error handling

Android Permissions

Add to `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

Cost Estimation

Approximate costs (may vary by region): - ESP8266 NodeMCU: \$3-5 - Relay modules (3x): \$6-9 - Current sensors (3x): \$9-15 - Power supply: \$5-10 - Wires and components: \$5-10 - Enclosure: \$5-15 - **Total: ~\$35-65**

Safety Reminders

IMPORTANT SAFETY NOTES:

1. **Always disconnect power** before making connections
 2. **Use proper wire gauges** for AC current
 3. **Install fuses** for overcurrent protection
 4. **Use proper enclosures** to prevent electric shock
 5. **Test with low voltage first** if possible
 6. **Double-check all connections** before powering on
 7. **Never touch live wires** with bare hands
 8. **Use GFCI outlets** if available
 9. **Get professional help** if unsure about AC wiring
-

Future Enhancements

1. **Voice Control** - Integrate with Google Assistant/Alexa
 2. **Energy Monitoring** - Track daily/weekly consumption
 3. **Automation Rules** - If-then conditions
 4. **Multiple Users** - User authentication
 5. **Notifications** - Push notifications for events
 6. **Data Logging** - Store historical data
 7. **Web Dashboard** - Browser-based control
 8. **Mobile Widgets** - Quick control from home screen
-

Support & Resources

- **ESP8266 Documentation:** <https://arduino-esp8266.readthedocs.io/>
 - **Flutter Documentation:** <https://flutter.dev/docs>
 - **Arduino Forum:** <https://forum.arduino.cc/>
 - **Project GitHub:** (Your repository link)
-

Quick Start Checklist

- ☐ Gather all hardware components
- ☐ Install Arduino IDE and libraries
- ☐ Upload Arduino code to ESP8266
- ☐ Configure WiFi and note IP address
- ☐ Assemble circuit on breadboard
- ☐ Test relay control manually
- ☐ Connect sensors and test readings
- ☐ Install Flutter dependencies (`flutter pub get`)

- ☐ Update IP address in Flutter app
- ☐ Run Flutter app and test connection
- ☐ Test device control
- ☐ Test voltage monitoring
- ☐ Test timer functionality
- ☐ Move to permanent enclosure
- ☐ Final safety check

Good luck with your IoT project! Stay safe and have fun building!